ENTANGLED ECHO



Undertaken by:-
Abdul Hafeez SP23-BSE-047
Umer Jalal SP23-BSE-055

SUPERVISED BY:-
Ma'am Tooba Tehreem

COMSATS University Islamabad, WAH CAMPUS
SESSION 2023-2027

CONTENTS

CONTENTS **2**
LIST OF FIGURES **3**
LIST OF TABLES **4**
1 INTRODUCTION **5**
1.1 System Introduction 5
1.2 Background of the System 5
1.3 Objectives of the System 5
1.4 Significance of the System 5
2 OVERALL DESCRIPTION **7**
2.1 Product Perspective 7
2.2 Product Scope 7
2.3 Product Functionality 8
2.4 Users and Characteristics 8
2.5 Operating Environment 8
3 SPECIFIC REQUIREMENTS **10**
3.1 Functional Requirements 10
3.1.1 User Management & Authentication *10*
3.1.2 Data Collection & Normalization *10*
3.1.3 Content Analysis *10*
3.1.4 Quantum-Inspired Optimization *11*
3.1.5 AI Recommendations & Task Decomposition *11*
3.1.6 Content Scheduling & Automation *11*
3.1.7 Analytics & Reporting *11*
3.1.8 Notification System *12*
3.2 Behaviour Requirements 12
3.3 External Interface Requirements 14
3.3.1 User Interfaces *14*
3.3.2 Software Interfaces *17*
3.3.3 Communications Interfaces *17*
4 OTHER NON-FUNCTIONAL REQUIREMENTS **18**
4.1 Performance Requirements 18
4.2 Safety and Security Requirements 18
4.3 Software Quality Attributes 18
4.3.1 Reliability *18*
4.3.2 Maintainability *18*
4.3.3 Portability *18*
4.3.4 Usability *18*
4.3.5 Scalability *19*
5 DESIGN DESCRIPTION **20**
5.1 Composite Viewpoint 20
5.2 Logical Viewpoint 20
5.3 Information Viewpoint 22
5.4 Interaction Viewpoint 22
5.5 State Dynamics Viewpoint 30
5.6 Algorithm Viewpoint 32
5.6.1 Algorithm 1: Quantum-Inspired Schedule Optimizer *33*
5.6.2 Algorithm 2: AI Recommendation Engine *33*

LIST OF FIGURES

Figure 2-1 : System architecture diagram 7
Figure 3-1 : Use Case Diagram for the System ..... **Error! Bookmark not defined.**
Figure 3-2 : Dashboard 14
Figure 3-3 : Analytics Page 14
Figure 3-4 : Recommendations Page 15
Figure 3-5 : Content calendar Page 15
Figure 3-6 : Notifications Page 16
Figure 3-7 : Settings Page 16
Figure 5-1 : Package diagram 20
Figure 5-2 : Class Diagram 21
Figure 5-3 : Entity Relationship Diagram 22
Figure 5-4 : UserManagement Auth 23
Figure 5-5 : Data collection and normalization 24
Figure 5-6 : Content analysis 25
Figure 5-7 : Scheduling 26
Figure 5-8 : Recommendation and task decomposition 27
Figure 5-9 : Content automation 28
Figure 5-10 : Analytics and Reporting 29
Figure 5-11 : Notifications 30
Figure 5-12 : State machine diagram 31
Figure 5-13 : Activity diagram 32

LIST OF TABLES

Table 1 : Users and Characteristics....8
Table 2 : User Management & Authentication....10
Table 3 : Data Collection & Normalization....10
Table 4 : Content anaylsis....11
Table 5 : Quantum-Inspired Optimization....11
Table 6 : AI Recommendations & Task Decomposition....11
Table 7 : Content Scheduling & Automation....11
Table 8 : Analytics & Reporting....12
Table 9 : Notification System....12
Table 10 : Use Case Descriptions and Actors....12

1 Introduction

1.1 System Introduction

Entangled Echo is an AI-powered web platform designed to help social media creators and small businesses optimize their content strategy. It leverages machine learning and quantum-inspired optimization techniques to analyze historical engagement data, audience behavior, and posting patterns, then recommends the best content types, captions, hashtags, and publishing schedules to maximize organic reach and audience growth.

The platform is delivered as a responsive web application built on Next.js and integrates directly with major social media APIs (Instagram, Facebook) to collect live engagement metrics and automate post scheduling. Entangled Echo differs from conventional scheduling tools by replacing rule-based planning with data-driven, continuously learning recommendations.

1.2 Background of the System

The social media landscape has evolved into one of the primary channels for brand communication, customer acquisition, and community building. Existing tools such as Hootsuite, Buffer, and Later offer content scheduling and basic analytics dashboards. However, these platforms rely heavily on manual planning: the creator must still decide what to post, when to post, and how to structure content. Analytical insights are retrospective rather than predictive, and none of these tools incorporate advanced optimization algorithms that could automatically surface the optimal content strategy for a specific account.

Entangled Echo addresses this gap by combining AI-based content analysis with quantum-inspired combinatorial optimization. While classical optimization algorithms struggle with the exponential search space of possible posting schedules and content combinations, quantum-inspired heuristics (such as simulated quantum annealing and variational optimization) can approximate optimal solutions efficiently. By integrating these techniques into an accessible web platform, Entangled Echo brings enterprise-grade content intelligence to independent creators and small businesses that lack dedicated marketing teams.

1.3 Objectives of the System

- Optimize social media content and posting schedules using AI-driven recommendations and quantum-inspired optimization algorithms.
- Reduce manual content planning effort through intelligent automation of scheduling, caption generation, and hashtag suggestions.
- Increase audience engagement and reach by predicting optimal posting windows and content formats per platform.
- Provide data-driven insights through interactive analytics dashboards covering engagement trends, audience demographics, and competitor analysis.
- Automate post publishing by integrating with social media platform APIs (Instagram Graph API, Facebook Graph API).
- Decompose high-level content strategies into actionable, atomic tasks (posts, stories, campaigns) using AI task planning.
- Ensure data security and privacy through encryption, role-based access control, and secure API credential management.

1.4 Significance of the System

Entangled Echo addresses a critical pain point experienced by millions of independent content creators and small businesses globally: the lack of intelligent, automated tools for content strategy optimization. The significance of the system spans several dimensions:

- Economic Impact: By automating content planning and improving engagement rates, Entangled Echo directly contributes to higher conversion rates and reduced marketing overhead for small businesses.

- Technological Novelty: The application of quantum-inspired optimization to social media scheduling represents a novel contribution at the intersection of AI, optimization research, and digital marketing.
- Accessibility: The platform democratizes access to advanced analytics and AI-based recommendations that are otherwise available only to large enterprises with dedicated data science teams.
- Research Potential: The system serves as a testbed for evaluating quantum-inspired algorithms in real-world combinatorial optimization problems, contributing to academic research in the field.
- Application Domains: Content optimization for social media creators; digital marketing automation for SMEs; automated campaign planning for e-commerce brands; engagement analytics for marketing agencies.

2 Overall Description

2.1 Product Perspective

Entangled Echo is a standalone, cloud-hosted web application. It does not replace social media platforms themselves but acts as an intelligent middleware layer that connects to platform APIs, processes engagement data, and feeds optimized recommendations back to the user. The system interacts with three primary external environments:

- Social Media Platforms: Instagram Graph API and Facebook Graph API provide read access to post metrics, audience insights, and write access for scheduled post publishing.
- AI & Optimization Engine: A Python backend service hosts the machine learning models (content analysis, engagement prediction) and the quantum-inspired optimization scheduler.
- User-Facing Web Application: A Next.js frontend delivers dashboards, recommendation interfaces, a content calendar, and a scheduling workflow to end users.

The system is self-contained in terms of user management; it does not depend on an existing organizational IT infrastructure and is designed to be deployable on standard cloud infrastructure (Supabase for database and auth, serverless functions for the backend).

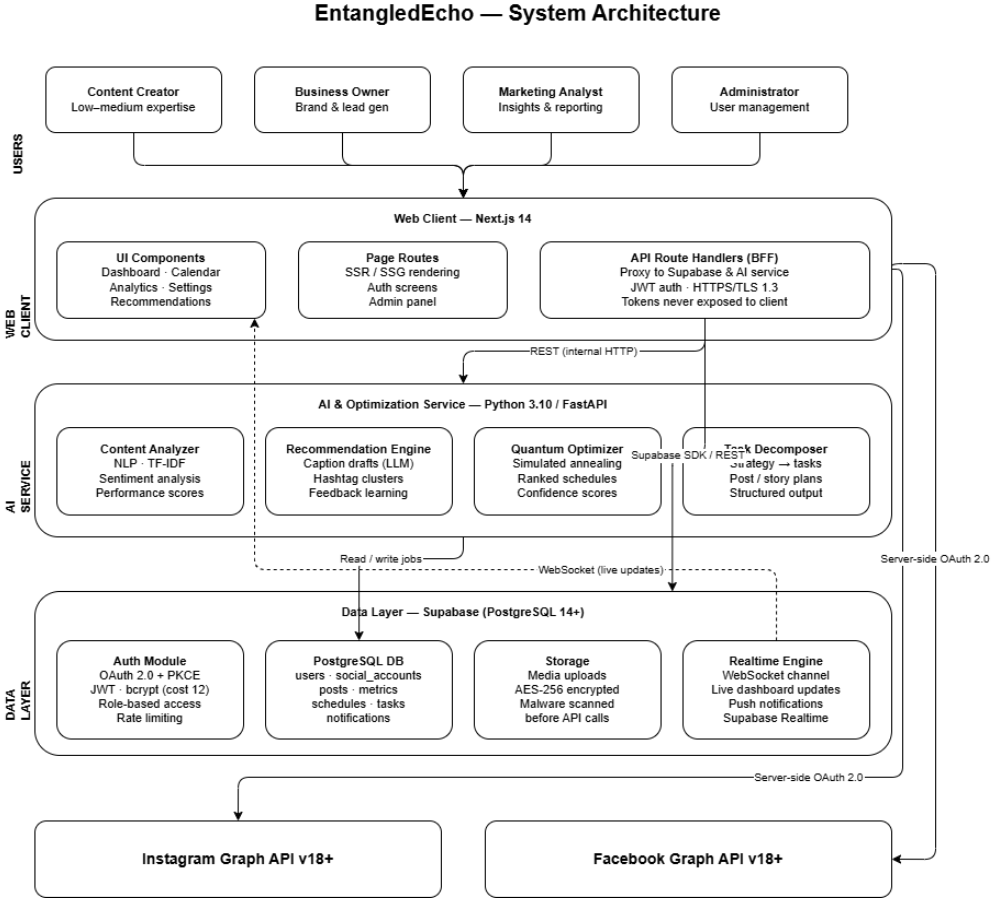


Figure 2-1 : System architecture diagram

2.2 Product Scope

Entangled Echo is scoped to: user account management; connection and data ingestion from Instagram and Facebook accounts; analysis of historical post performance; generation of content schedule recommendations; AI-assisted caption and hashtag suggestions; automated post scheduling via social media APIs; and visual analytics dashboards.

The following are explicitly out of scope for this version: support for TikTok, Twitter/X, LinkedIn, or YouTube; video editing or image generation; e-commerce transaction processing; multi-tenant agency workflows; and native mobile applications. These may be addressed in future iterations.

2.3 Product Functionality

- User Registration, Login & Profile Management with social media account linking.
- OAuth-based Social Media Account Connection (Instagram, Facebook).
- Automated Data Collection: ingestion and normalization of engagement metrics (likes, comments, shares, reach, impressions).
- Content Analysis: NLP-based analysis of captions, hashtags, media types, and post timing to identify performance patterns.
- Quantum-Inspired Schedule Optimization: generation of optimal weekly/monthly posting schedules.
- AI Content Recommendations: personalized suggestions for captions, hashtags, content types, and posting frequency.
- AI Task Decomposition: breaking high-level content strategies into structured, actionable post plans.
- Content Scheduling & Automation: calendar-based scheduling with automated publishing via social media APIs.
- Analytics Dashboard: visual reporting on engagement trends, top-performing content, audience demographics, and competitor benchmarks.
- Notification System: alerts for scheduled posts, engagement milestones, and AI-generated insights.
- Security & Privacy: JWT-based authentication, encrypted credential storage, and role-based access control.

2.4 Users and Characteristics

The system is designed for the following user types:

User Type	Technical Expertise	Primary Goals	Key Features Used
Content Creator	Low to Medium	Grow audience, maximize engagement	Recommendations, Scheduler, Analytics
Small Business Owner	Low	Brand awareness, lead generation	Campaign Planner, AI Tasks, Scheduler
Marketing Analyst	High	Insights, competitor analysis, reporting	Analytics Dashboard, Competitor Analysis
System Administrator	High	System management, user oversight	Admin Panel, User Management, Logs

Table 1 : Users and Characteristics

2.5 Operating Environment

- Client-Side: Any modern web browser (Chrome 110+, Firefox 110+, Edge 110+, Safari 16+) on desktop or mobile. No additional software installation required.
- Server-Side Frontend: Next.js 14+ on Node.js 18+. Supports SSR and SSG.

- Backend AI Services: Python 3.10+ hosting FastAPI endpoints for ML inference, data processing, and quantum-inspired optimization routines.
- Database & Authentication: Supabase (PostgreSQL 14+) with row-level security and OAuth-based user authentication.
- APIs: Instagram Graph API v18+, Facebook Graph API v18+. OAuth 2.0 for all third-party authorizations.
- Network: HTTPS for all client-server communication. REST and WebSocket protocols. Minimum 1 Mbps bandwidth recommended.
- Hardware: No specialized hardware required. All processing handled on standard cloud infrastructure.

3 Specific Requirements

3.1 Functional Requirements

3.1.1 User Management & Authentication

ID	Requirement Name	Detailed Description
FR-01	User Registration	Allow registration with a valid email and password.
FR-02	OAuth 2.0 Support	Support Google login as a third-party identity provider.
FR-03	Email Verification	Enforce verification before activating new accounts.
FR-04	Password Reset	Secure reset via time-limited email links.
FR-05	User Profiles	Maintain display name, bio, and connected social accounts.
FR-06	Role-Based Access	Implement Standard User and Administrator roles.
FR-07	Social Integration	Link/unlink Instagram and Facebook via OAuth.
FR-08	Token Security	Encrypt OAuth tokens and implement auto-refresh logic.

Table 2 : User Management & Authentication

3.1.2 Data Collection & Normalization

ID	Requirement Name	Detailed Description
FR-09	Metric Retrieval	Retrieve engagement metrics every 24 hours.
FR-10	Normalization	Convert platform data into a unified internal schema.
FR-11	Data Retention	Store historical data for at least 12 months.
FR-12	Sync Status	Display last successful sync timestamp on dashboard.

Table 3 : Data Collection & Normalization

3.1.3 Content Analysis

ID	Requirement Name	Detailed Description
FR-13	NLP Analysis	Extract topics, sentiment, and terms from captions.
FR-14	Media Classification	Classify by type: Image, Video, Carousel, Story, Reel.
FR-15	Performance Insights	Identify top content, hashtags, and optimal post times.
FR-16	Performance Scoring	Generate weighted engagement scores per post.

Table 4 : Content anaylsis

3.1.4 Quantum-Inspired Optimization

ID	Requirement Name	Detailed Description
FR-17	Schedule Generation	Generate weekly schedules using quantum-inspired heuristics.
FR-18	User Constraints	Support blackout periods, platform choice, and post caps.
FR-19	Ranked Alternatives	Provide alternative schedules if conflicts arise.
FR-20	Confidence Metrics	Show confidence scores for each recommendation.

Table 5 : Quantum-Inspired Optimization

3.1.5 AI Recommendations & Task Decomposition

ID	Requirement Name	Detailed Description
FR-21	Content Templates	Suggest hashtags and caption templates based on history.
FR-22	Task Decomposition	Break down high-level goals into specific post tasks.
FR-23	AI Drafts	Generate editable AI-written caption drafts.
FR-24	Machine Learning	Learn from accept/reject actions to improve future results.

Table 6 : AI Recommendations & Task Decomposition

3.1.6 Content Scheduling & Automation

ID	Requirement Name	Detailed Description
FR-25	Calendar Interface	Drag-and-drop UI for manual and AI scheduling.
FR-26	Auto-Publishing	Direct publishing to IG and FB via official APIs.
FR-27	Retry Mechanism	3 retries with exponential back-off for failures.
FR-28	Failure Alerts	Immediate user notification for publishing failures.

Table 7 : Content Scheduling & Automation

3.1.7 Analytics & Reporting

ID	Requirement Name	Detailed Description
FR-29	Trend Visualization	Daily, weekly, and monthly engagement charts.
FR-30	Audience Breakdown	Demographics: age, gender, and location.
FR-31	Data Export	Export analytical data to CSV format.
FR-32	Competitive Analysis	Comparison view for specified

		competitors.

Table 8 : Analytics & Reporting

3.1.8 Notification System

ID	Requirement Name	Detailed Description
FR-33	In-App Alerts	Alerts for posts, confirmations, and failures.
FR-34	Email Digests	Weekly summaries and major engagement milestones.
FR-35	Customization	Allow users to opt-out or customize alert preferences.

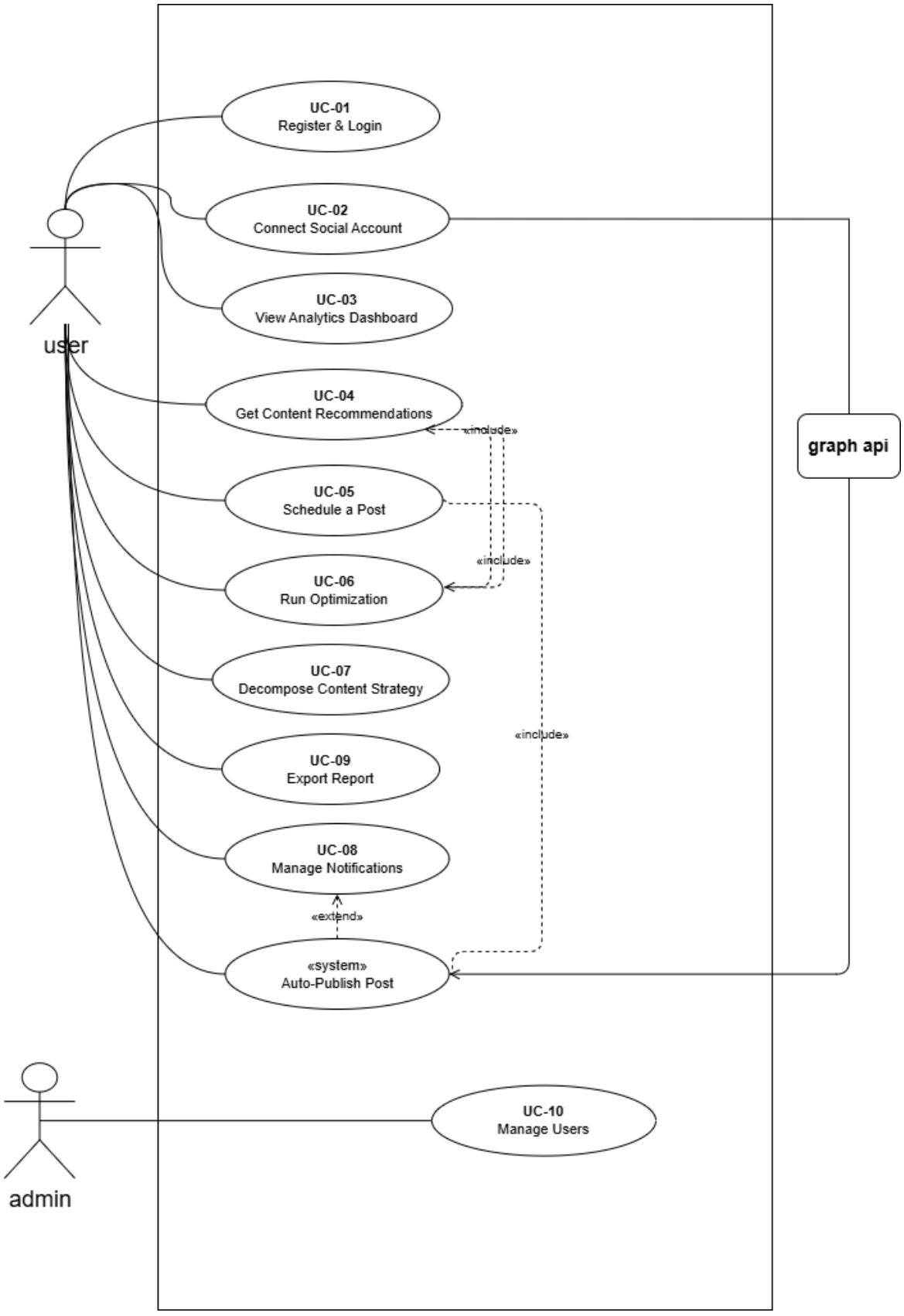
Table 9 : Notification System

3.2 Behaviour Requirements

The following use cases encapsulate the primary interactions between users and EntangledEcho. The Use Case Diagram below illustrates all actors and their relationships with system use cases.

UC-ID	Use Case Name	Actor(s)	Brief Description
UC-01	**Register & Login**	User	User creates an account, verifies email, and logs in with credentials or Google OAuth.
UC-02	**Connect Social Account**	User	User authorizes Entangled Echo to access their Instagram/Facebook account via OAuth.
UC-03	**View Analytics Dashboard**	User	User views engagement metrics, trends, and top-performing content for connected accounts.
UC-04	**Get Content Recommendations**	User	System analyzes historical data and presents AI-generated content and schedule recommendations.
UC-05	**Schedule a Post**	User	User selects a time slot in the content calendar, attaches media, and confirms scheduling.
UC-06	**Run Optimization**	User	User triggers quantum-inspired schedule optimization with optional constraints.
UC-07	**Decompose Content Strategy**	User	User enters a high-level goal; system returns a structured task list of posts and stories.
UC-08	**Manage Notifications**	User	User configures notification preferences for posts, summaries, and milestones.
UC-09	**Export Report**	User	User selects a date range and downloads engagement data as a CSV file.
UC-10	**Manage Users**	Admin	Administrator views, suspends, or removes user accounts via the admin panel.

Table 10 : Use Case Descriptions and Actors



3-1 : Use Case Diagram for the System

3.3 External Interface Requirements

3.3.1 User Interfaces

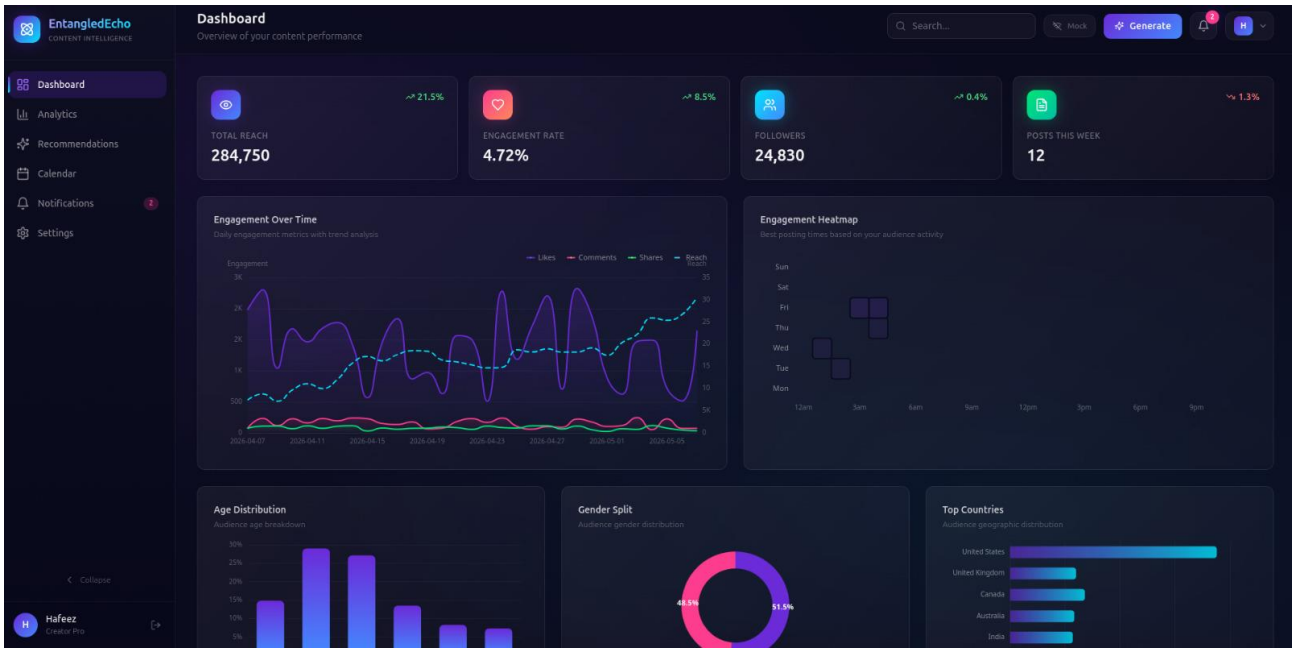


Figure 3-2 : Dashboard

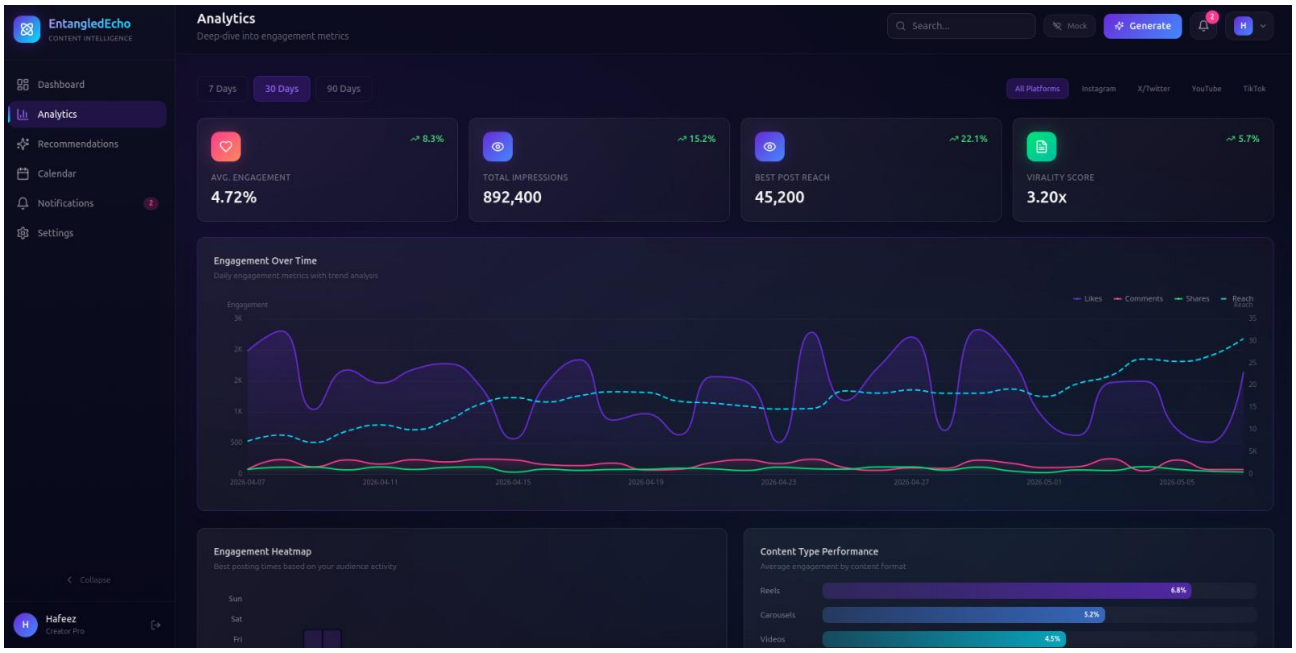


Figure 3-3 : Analytics Page

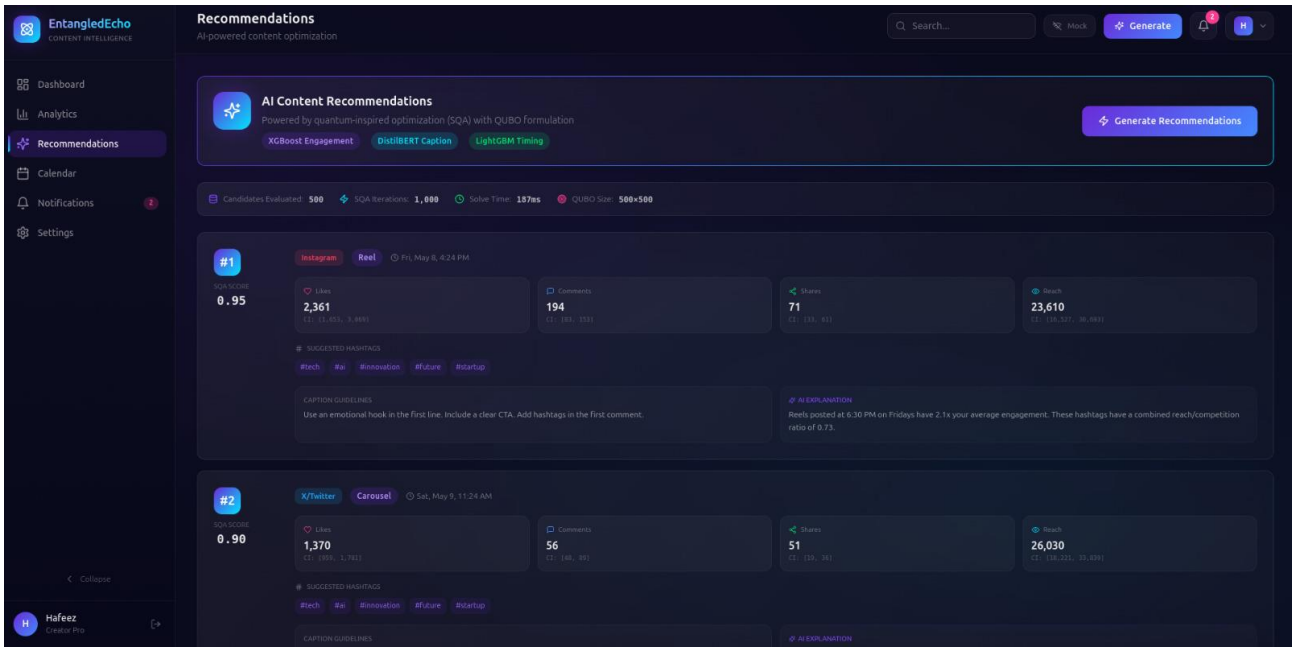


Figure 3-4 : Recommendations Page

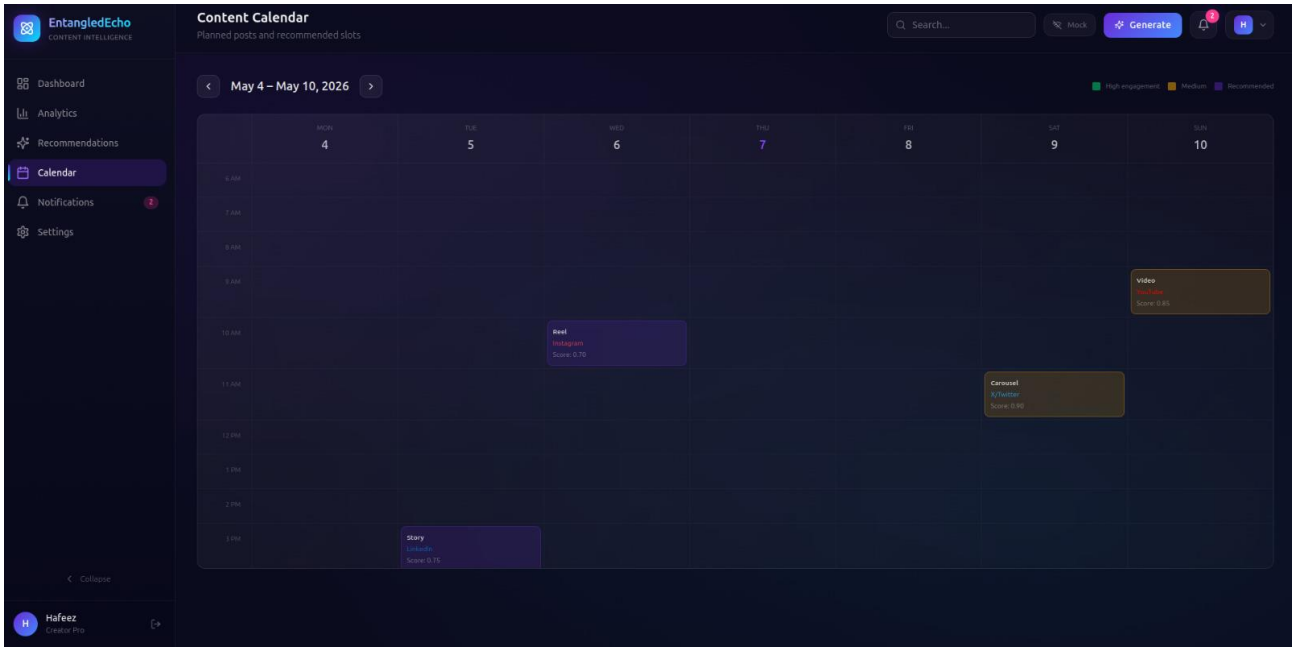


Figure 3-5 : Content calendar Page

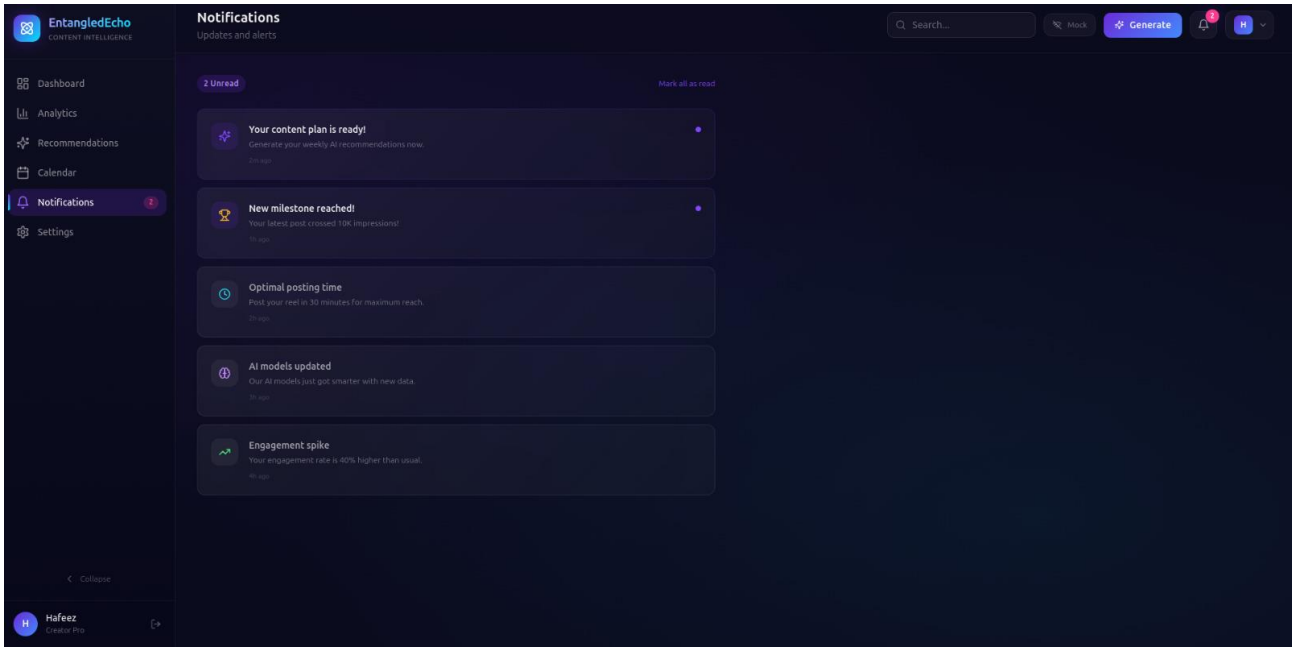


Figure 3-6 : Notifications Page

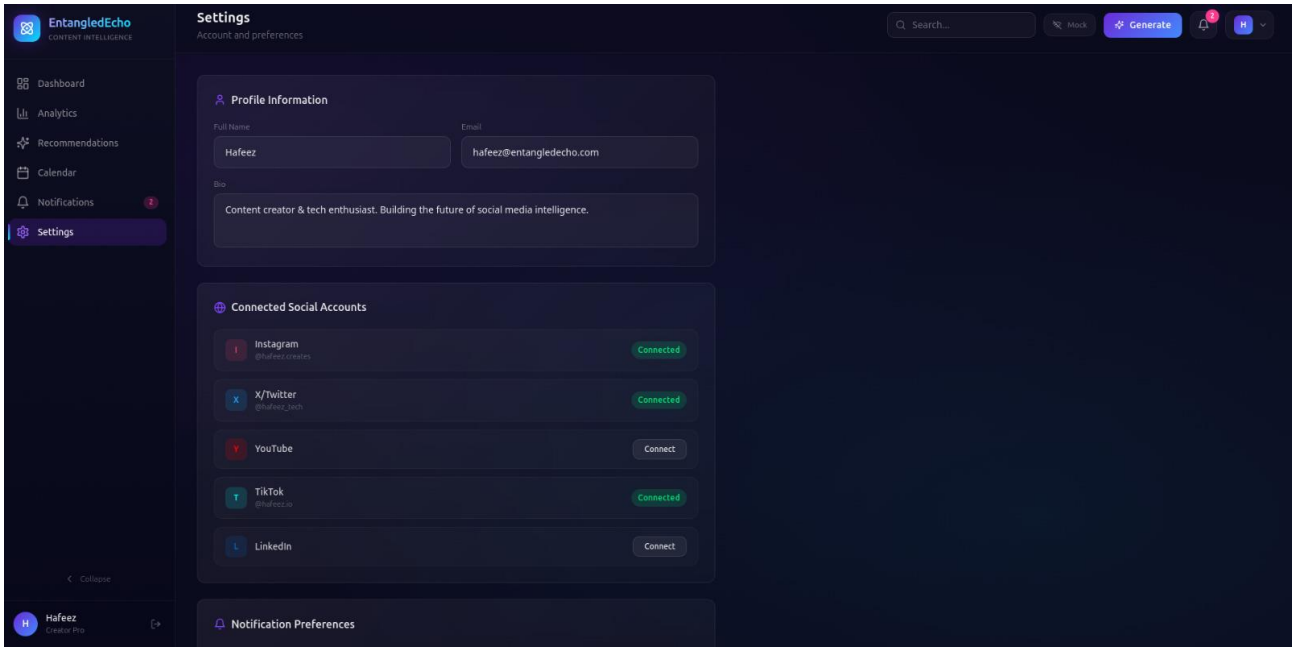


Figure 3-7 : Settings Page

3.3.2 Software Interfaces

- Instagram Graph API (v18+): Used for reading post insights, audience data, and publishing media. Requires: instagram_basic, instagram_content_publish, instagram_manage_insights.
- Facebook Graph API (v18+): Used for reading page insights and scheduling page posts. Requires: pages_read_engagement, pages_manage_posts.
- Supabase REST API & Realtime: Used for all CRUD operations on the PostgreSQL database. Row-Level Security policies enforce data isolation between user accounts.
- Python FastAPI Backend: Internal REST API exposing endpoints for ML inference, optimization routines, and data pipelines.
- Next.js App Router API Routes: Serve as BFF (Backend for Frontend) endpoints, proxying requests to Supabase and the Python service.

3.3.3 Communications Interfaces

All communications between the client browser and server use HTTPS (TLS 1.3) to ensure data in transit is encrypted. REST APIs use JSON payloads with standard HTTP status codes. WebSocket connections (via Supabase Realtime) are used for live dashboard updates and notification delivery. Social media API calls are made server-side to prevent exposure of access tokens to the client. Email notifications are delivered via a TLS-secured transactional email service. OAuth 2.0 with PKCE extension is enforced for all third-party account authorization flows.

4 Other Non-functional Requirements

4.1 Performance Requirements

1. All dashboard page loads shall complete within 3 seconds on a standard broadband connection (10 Mbps) under normal load.
2. Any REST API endpoint shall respond within 500 ms for standard CRUD operations under a load of up to 100 concurrent users.
3. The quantum-inspired schedule optimization job shall complete within 10 seconds for a 30-day horizon with up to 5 connected accounts.
4. AI caption and hashtag recommendations shall be returned within 5 seconds of user request.
5. Scheduled post publishing shall be triggered within a 60-second window of the user-defined time.
6. The analytics data sync job shall complete within 5 minutes for an account with 500 historical posts.

4.2 Safety and Security Requirements

1. All user passwords shall be hashed using bcrypt with a minimum cost factor of 12 before storage. Plain-text passwords shall never be logged or stored.
2. OAuth access tokens and refresh tokens shall be encrypted at rest using AES-256 before being persisted to the database.
3. All API endpoints that modify data shall require a valid JWT access token with appropriate role claims.
4. Row-Level Security (RLS) policies in Supabase shall ensure that users can only read and write their own data records.
5. The system shall rate-limit authentication endpoints to a maximum of 10 failed attempts per IP per 15-minute window, after which the IP is temporarily blocked.
6. Social media API credentials shall be stored in server-side environment variables and never exposed to the client browser.
7. All user-uploaded media shall be scanned for malicious content before being passed to social media APIs.
8. The admin panel shall be accessible only to accounts with the Administrator role and shall require re-authentication for destructive operations.

4.3 Software Quality Attributes

4.3.1 Reliability

Entangled Echo shall maintain 99.5% uptime for core features (dashboard, scheduler, analytics). Scheduled post publishing is a time-sensitive operation; it shall implement automatic retry (up to 3 attempts with exponential back-off) and alert the user upon final failure. Database operations shall use transactions to prevent partial writes that could corrupt engagement records.

4.3.2 Maintainability

The codebase shall follow a modular, feature-based directory structure separating UI components, API routes, service logic, and data access layers. Each module shall be independently testable. The Python backend shall expose versioned API endpoints (e.g., /api/v1/) so that future breaking changes can be introduced without disrupting existing clients. All environment-specific configuration shall be managed via environment variables, with no hard-coded secrets or URLs.

4.3.3 Portability

The web application shall function correctly on all major modern browsers (Chrome, Firefox, Edge, Safari) without browser-specific polyfills beyond standard Next.js defaults. The backend Python service shall be containerizable (Dockerfile provided) for deployment on any OCI-compliant container platform. The database schema shall be migration-controlled (via Supabase migrations) to support reproducible environment setup.

4.3.4 Usability

The platform shall conform to WCAG 2.1 Level AA accessibility guidelines for color contrast, keyboard navigation, and screen-reader compatibility. First-time users shall be guided through a step-by-step onboarding flow that connects their social accounts and explains core features within a session of under 10 minutes. All error messages shall be written in plain, actionable language; technical error codes shall appear only in collapsible detail panels.

4.3.5 Scalability

The architecture shall support horizontal scaling of the Next.js frontend via stateless deployment on a CDN-enabled edge network. The Python AI service shall support concurrent request handling via asynchronous request processing (FastAPI + async/await). The PostgreSQL database shall be designed with appropriate indexing on high-query columns (user_id, account_id, post_date) to sustain query performance as the dataset grows to millions of records.

5 Design Description

5.1 Composite Viewpoint

The system is organized into three top-level packages: (1) Web Client (Next.js) — containing UI Components, Page Routes, and API Route Handlers; (2) AI & Optimization Service (Python/FastAPI) — containing the Content Analyzer, Recommendation Engine, Quantum Optimizer, and Task Decomposer; and (3) Data Layer (Supabase) — containing the Auth module, PostgreSQL database, Storage, and Realtime engine. External dependencies include the Instagram Graph API and Facebook Graph API. The Web Client communicates with the Data Layer via REST (Supabase client SDK) and with the AI Service via internal HTTP calls from API Route Handlers.

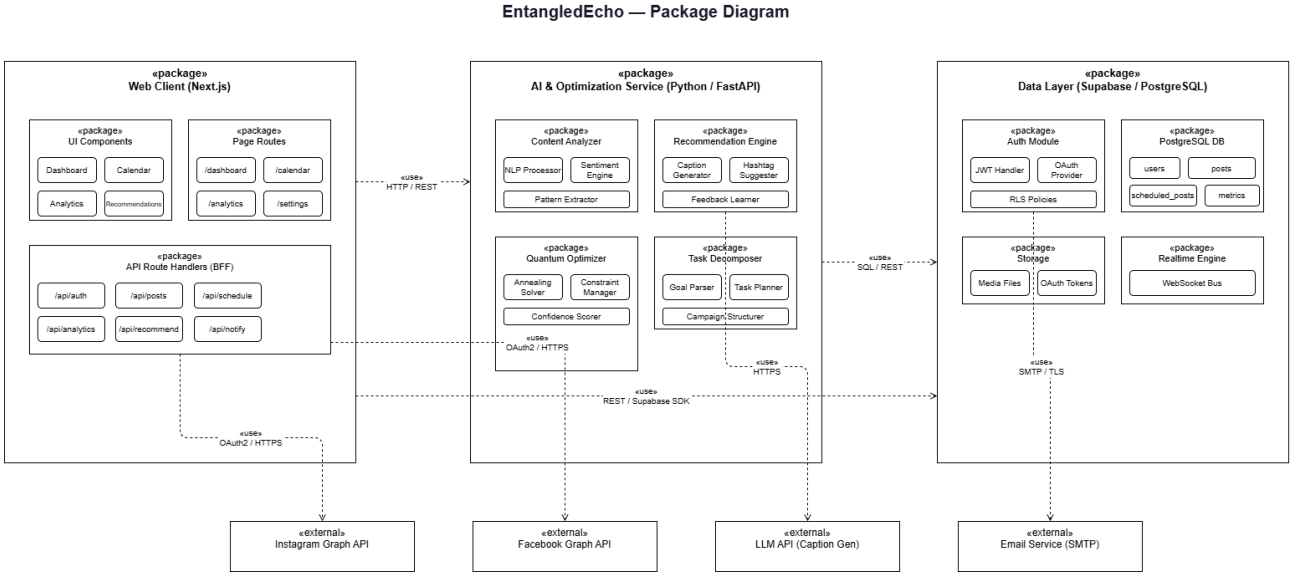


Figure 5-1 : Package diagram

5.2 Logical Viewpoint

The Class Diagram below defines the static structure of EntangledEcho's core domain model. It covers ten primary classes: User, SocialAccount, Post, EngagementMetric, ScheduledPost, Recommendation, OptimizationJob, Notification, ContentStrategy, and Task. Associations, multiplicities, and key attributes/methods are shown.

Figure 5.1 — Class Diagram: EntangledEcho (UML 2.5)

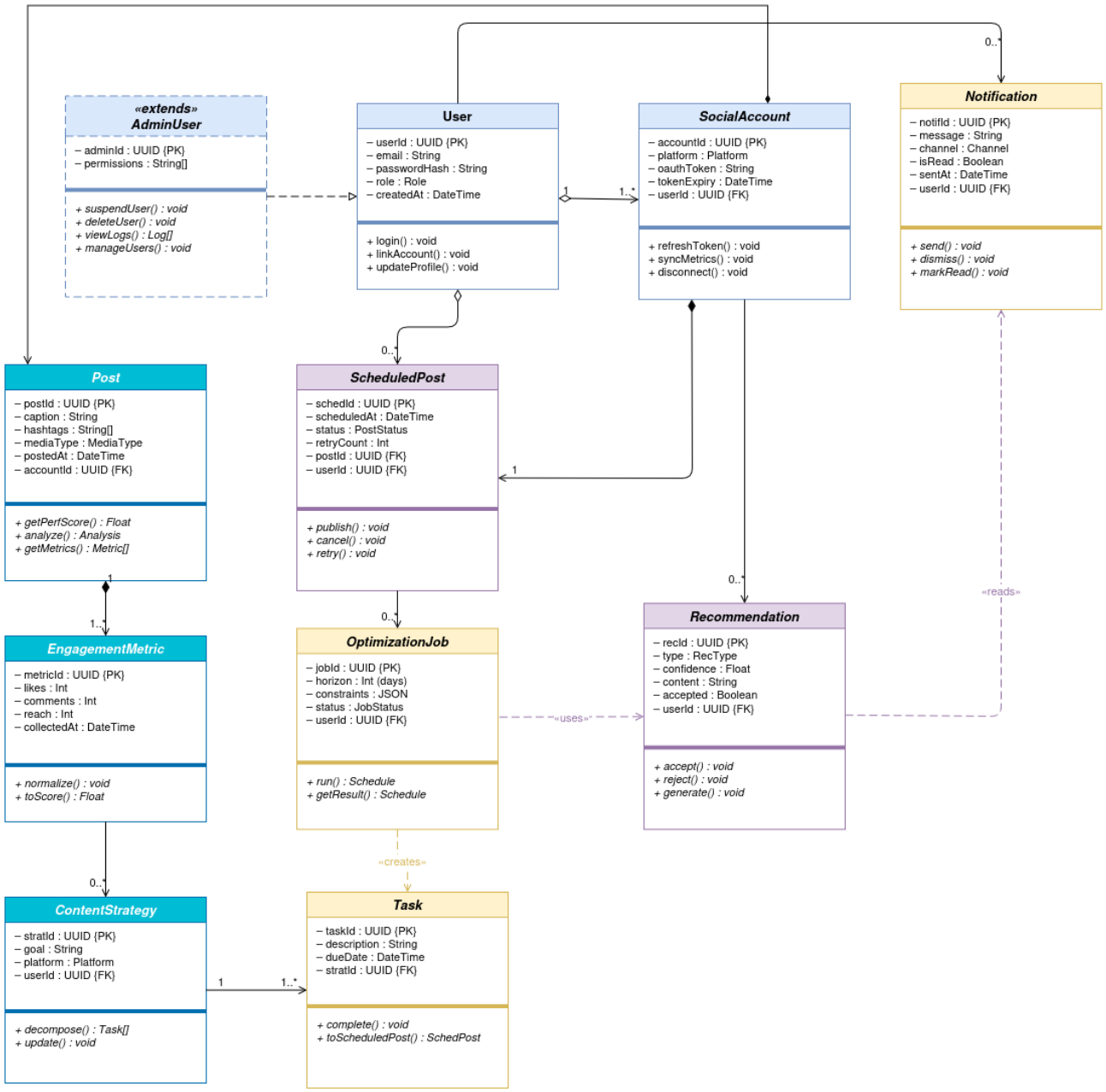


Figure 5-2 : Class Diagram

5.3 Information Viewpoint

The ERD below describes the persistent data model stored in the Supabase PostgreSQL database. Core entities include: users, social_accounts, posts, engagement_metrics, scheduled_posts, recommendations, optimization_jobs, tasks, and notifications. Primary keys (PK), foreign keys (FK), and cardinality relationships (1:1, 1:N) are shown. Row-Level Security is enforced at the database level on all user-scoped tables.

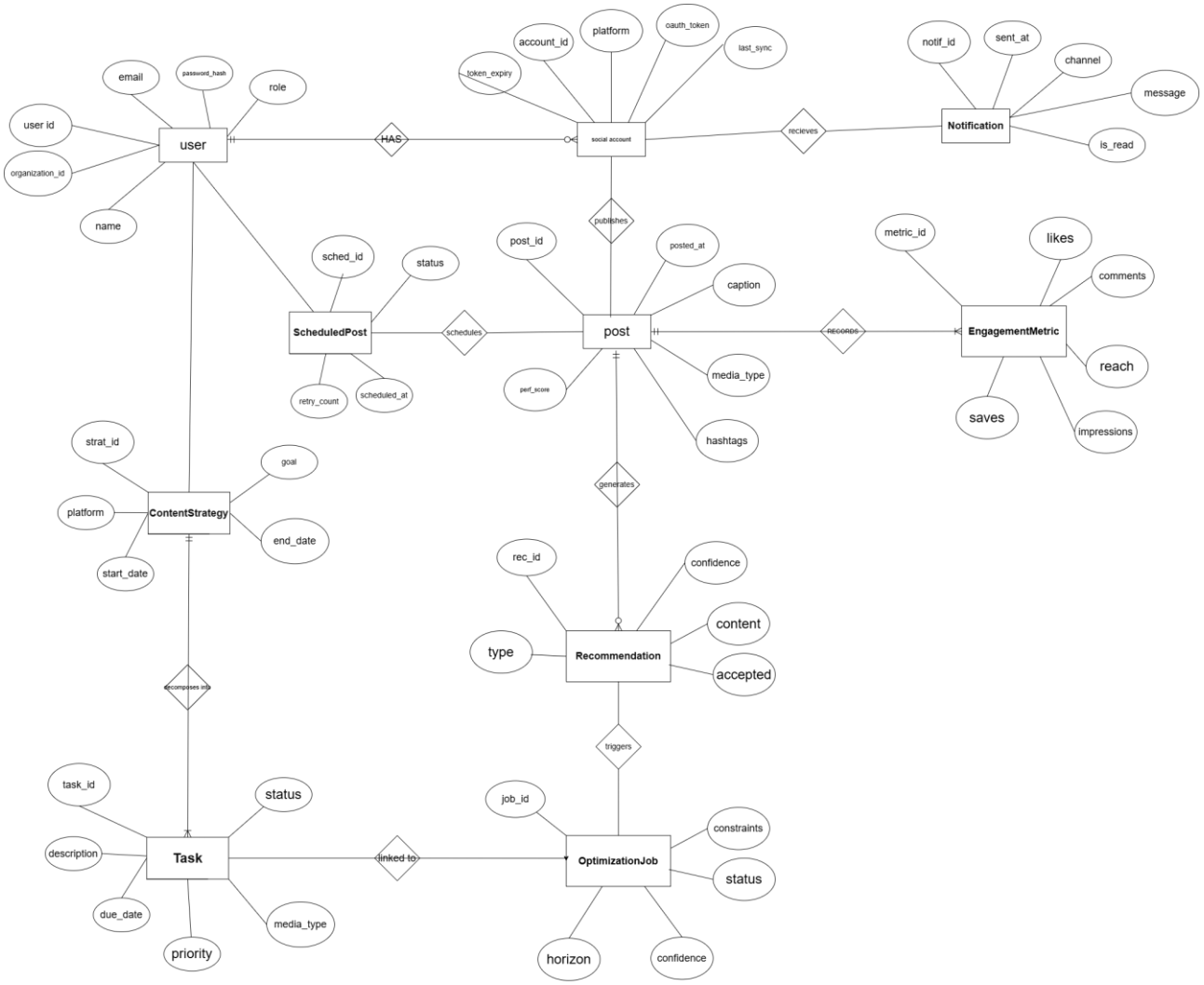


Figure 5-3 : Entity Relationship Diagram

5.4 Interaction Viewpoint

The Sequence Diagrams below of major use cases.

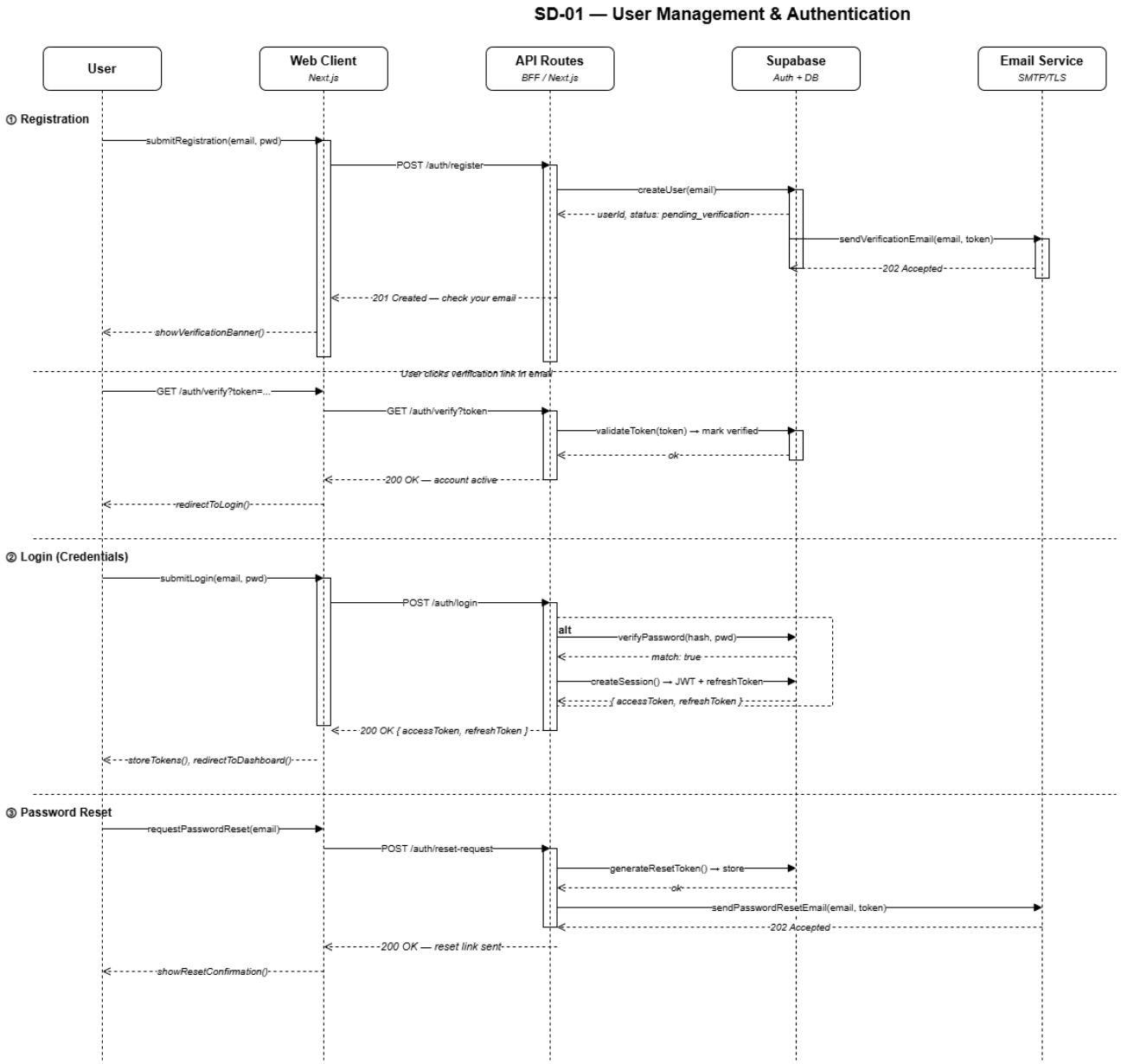


Figure 5-4 : UserManagement Auth

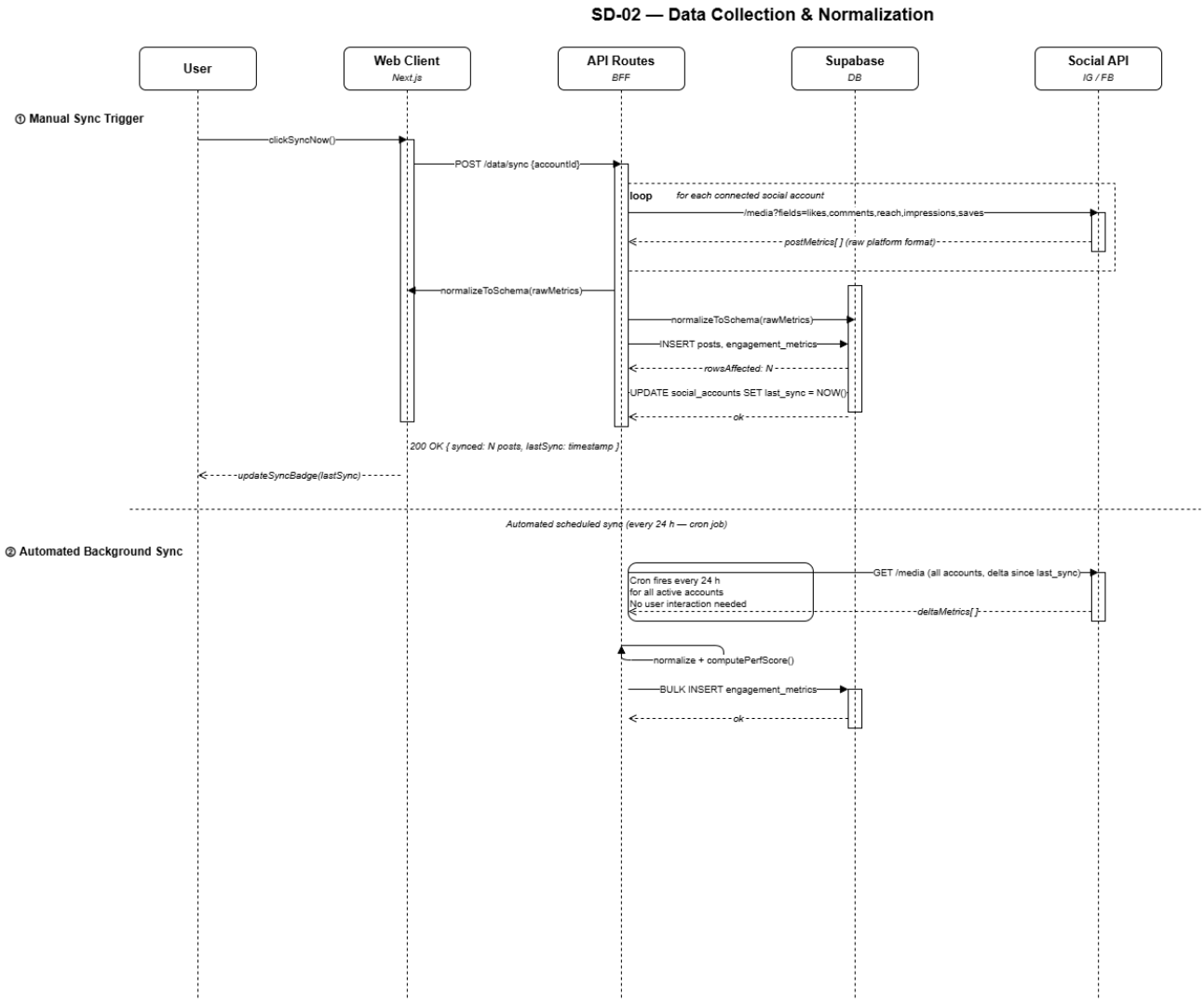


Figure 5-5 : Data collection and normalization

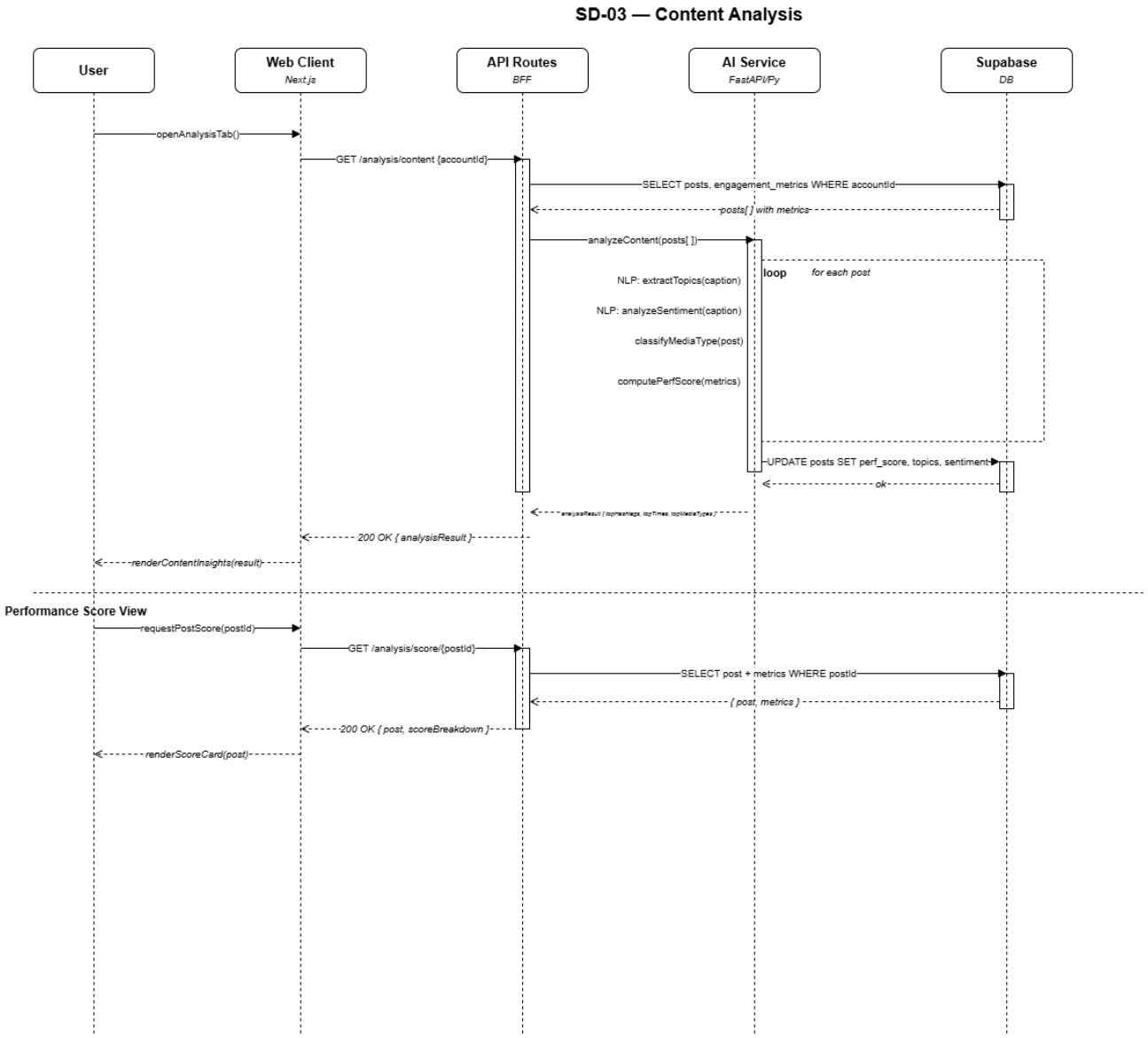


Figure 5-6 : Content analysis

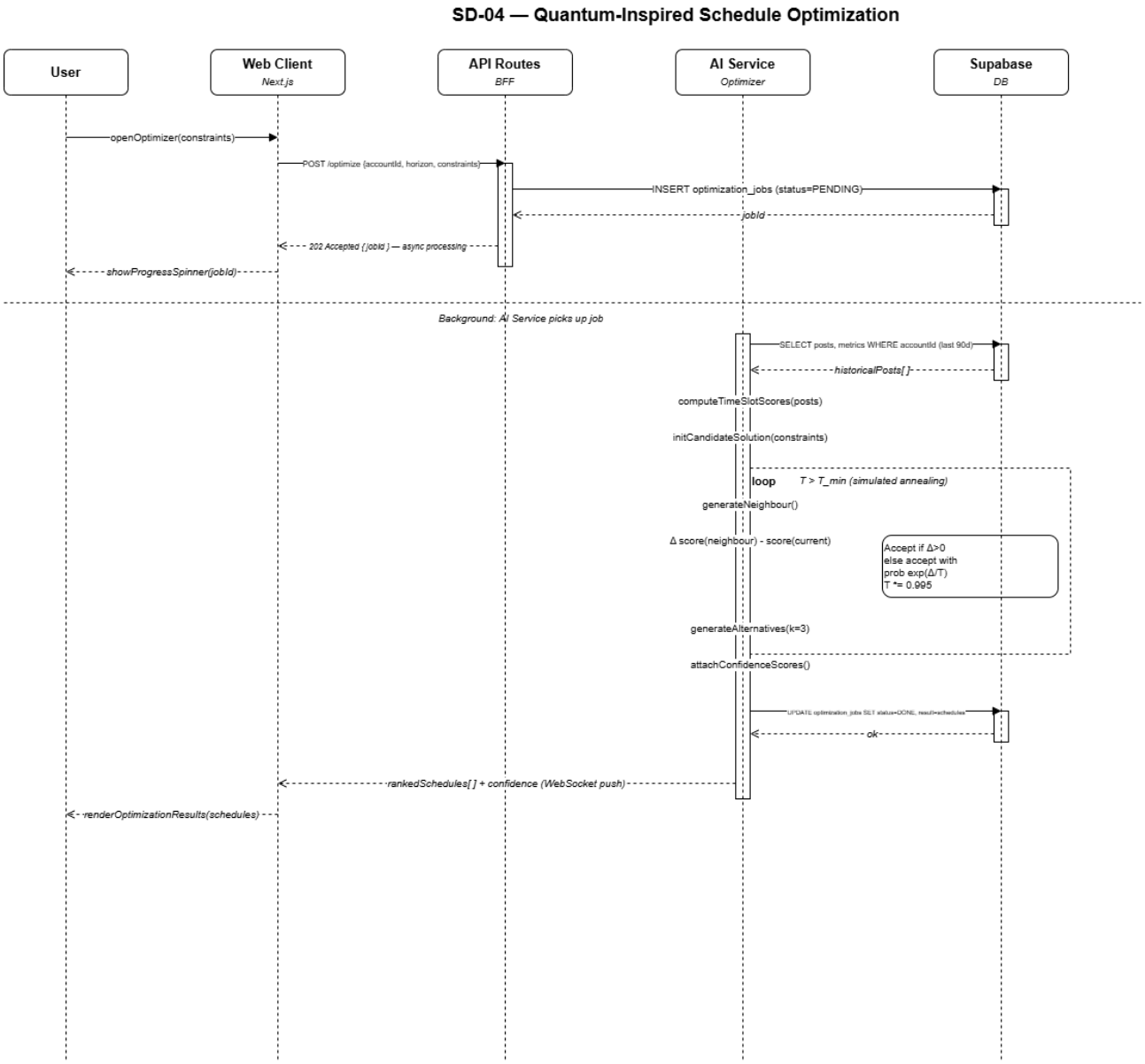


Figure 5-7 : Scheduling

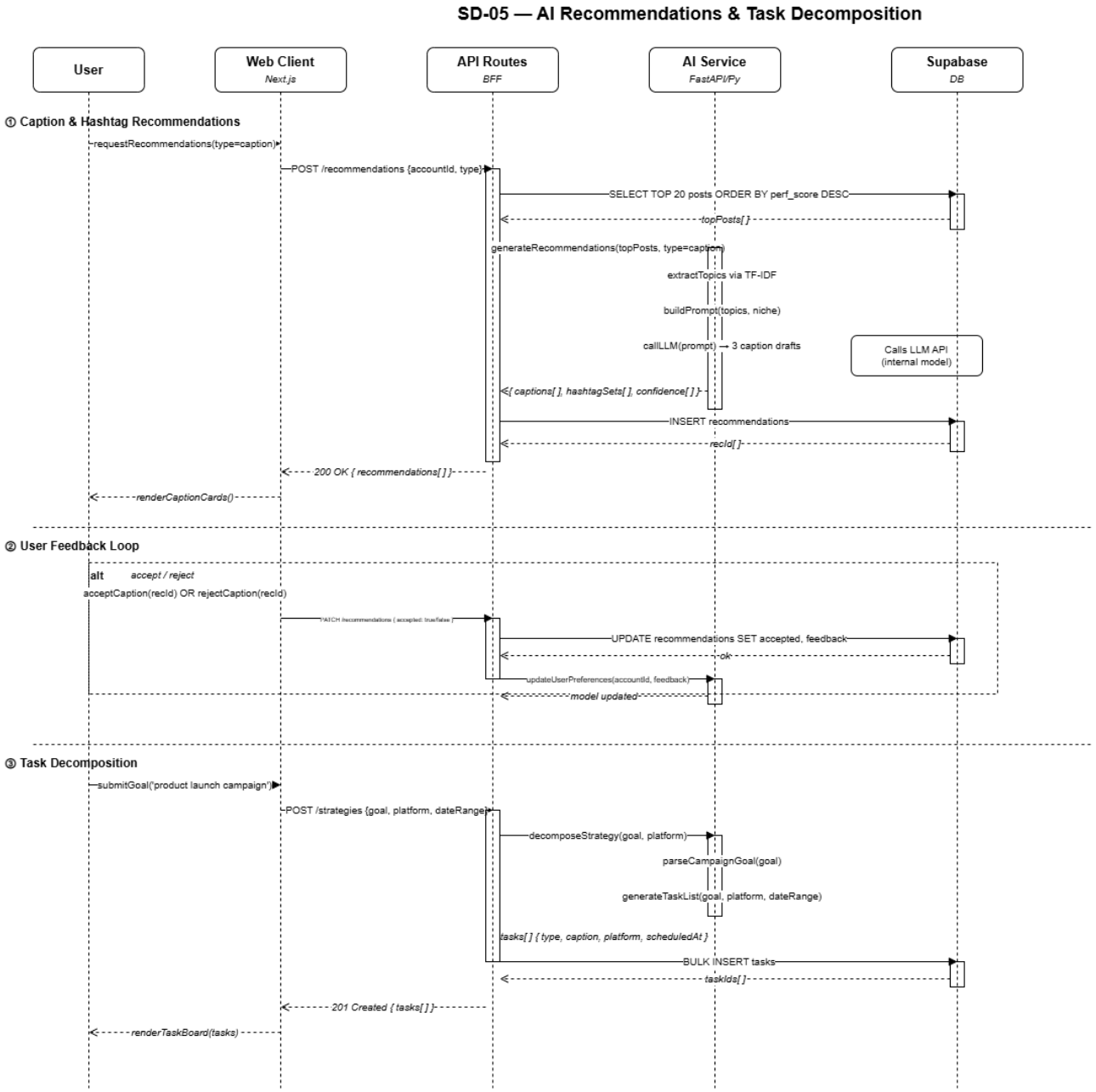


Figure 5-8 : Recommendation and task decomposition

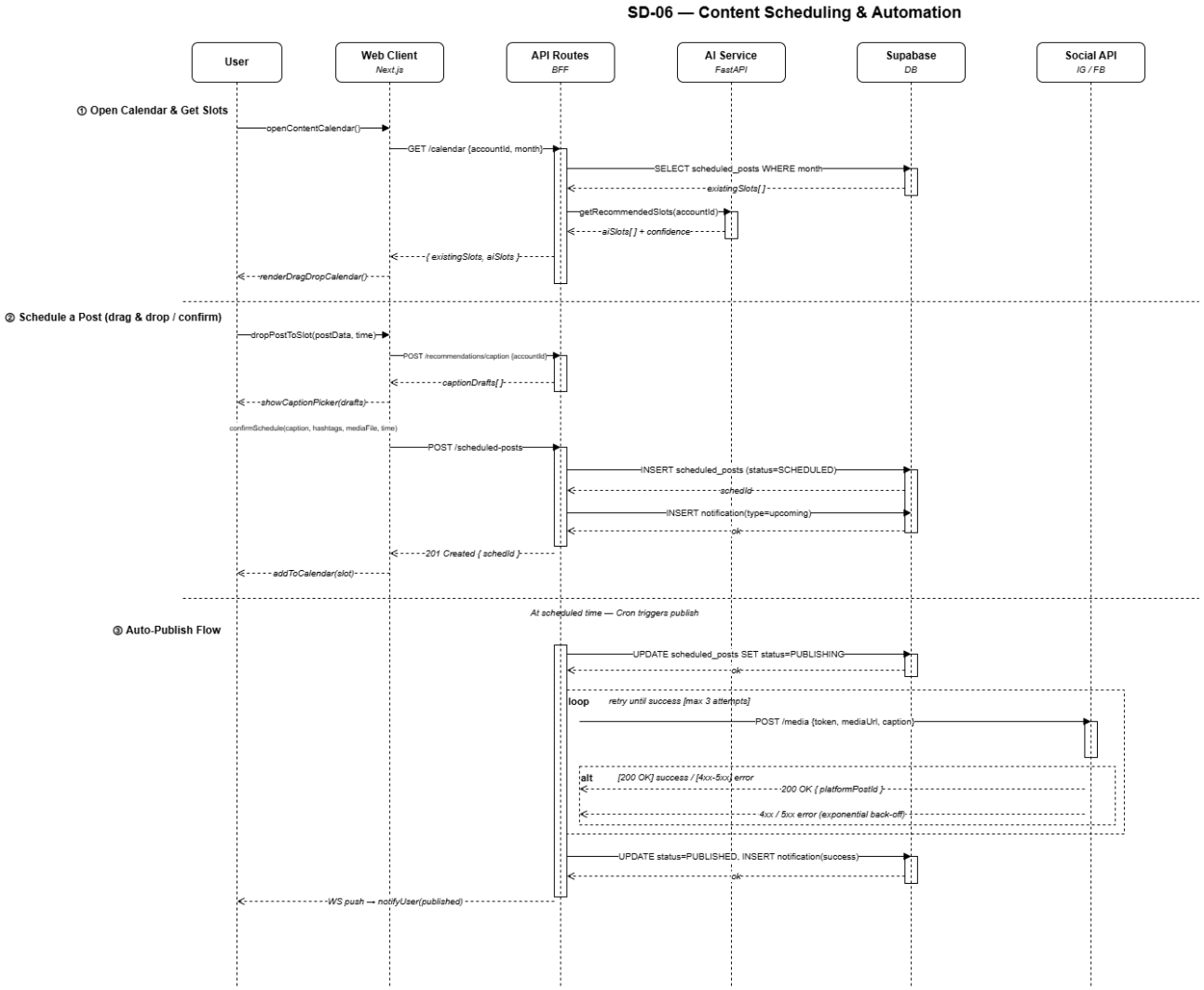


Figure 5-9 : Content automation

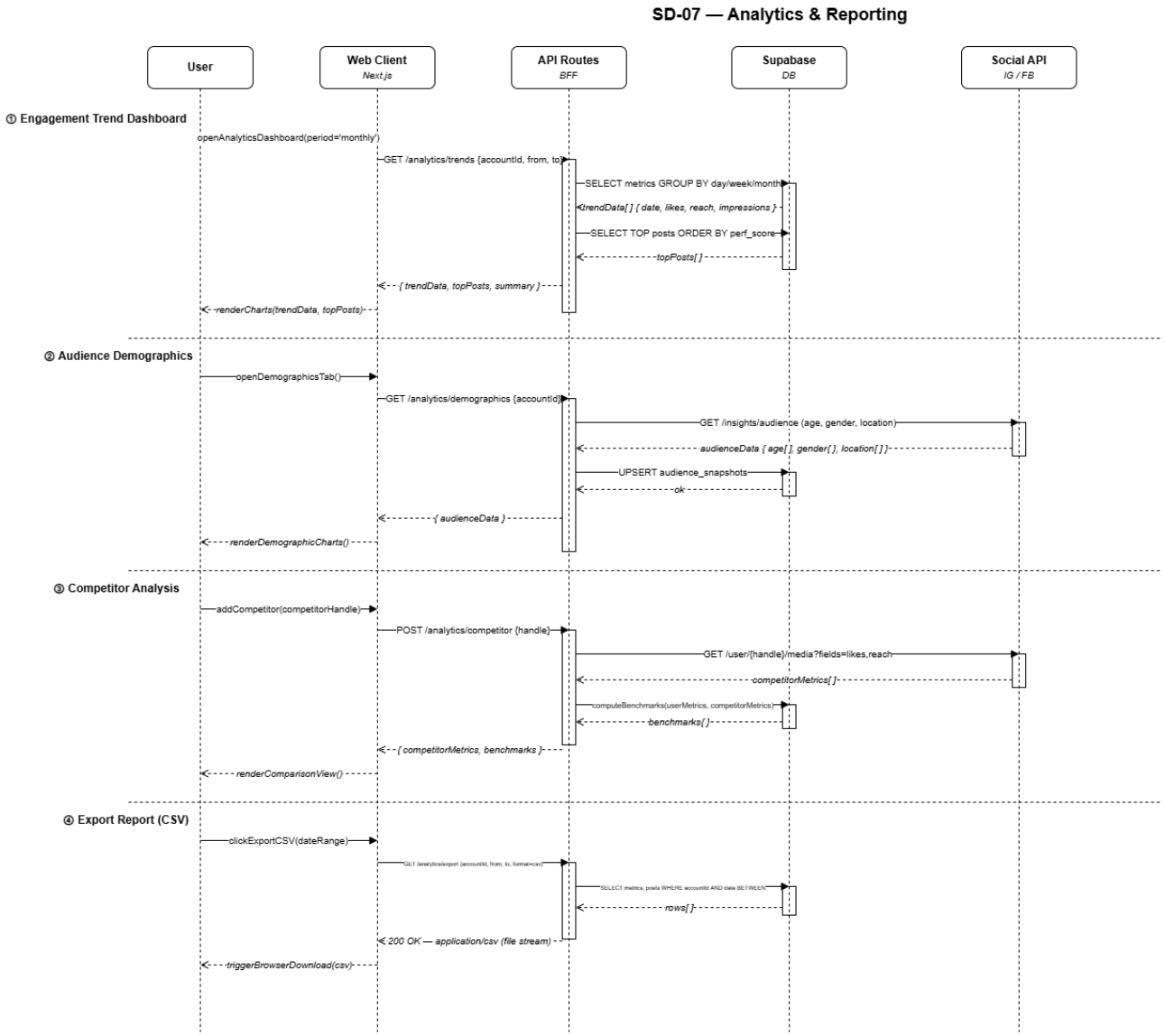


Figure 5-10 : Analytics and Reporting

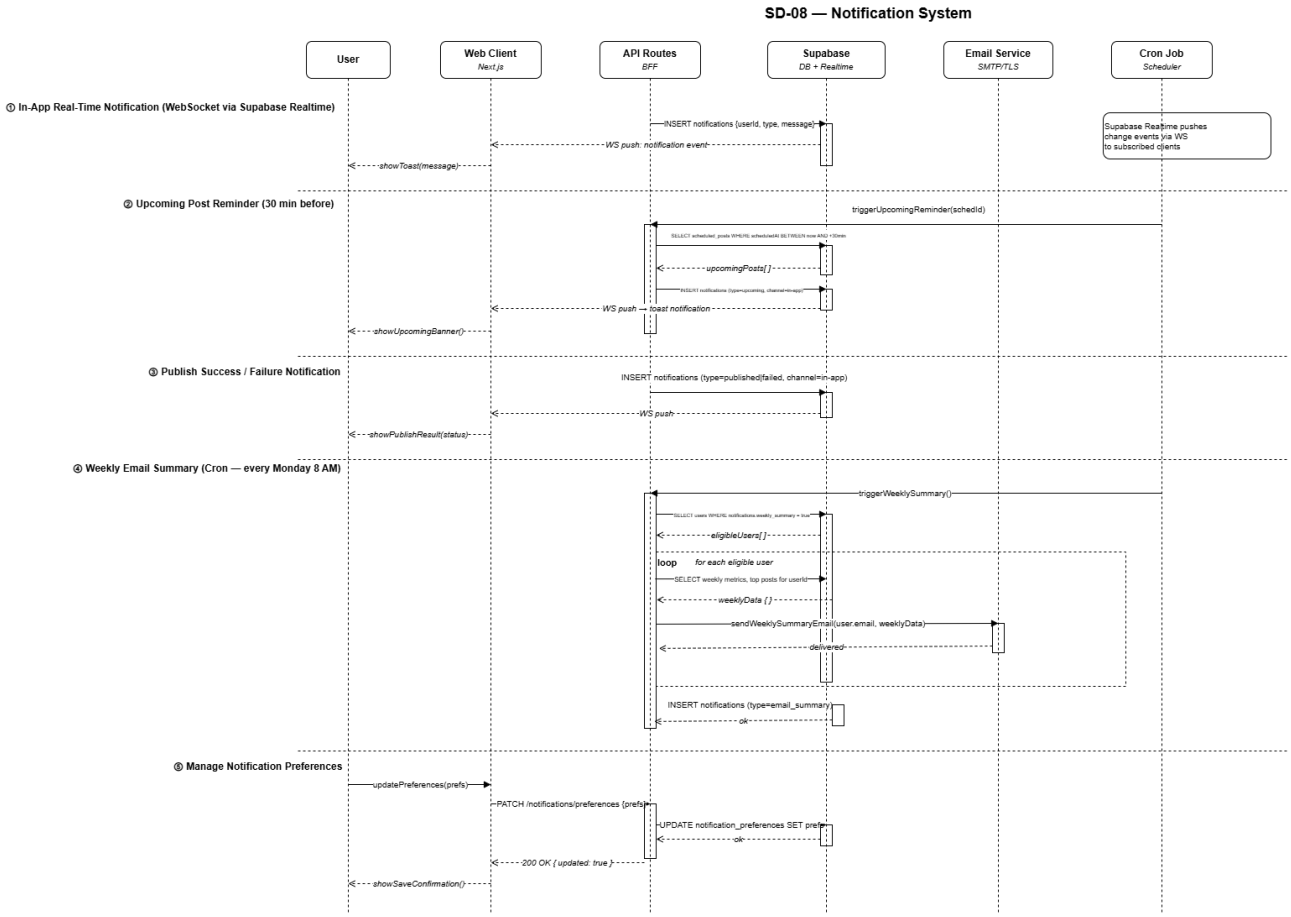


Figure 5-11 : Notifications

5.5 State Dynamics Viewpoint

The State Machine Diagram and State Transition Table for the ScheduledPost entity are described below. A ScheduledPost object transitions through the following states:

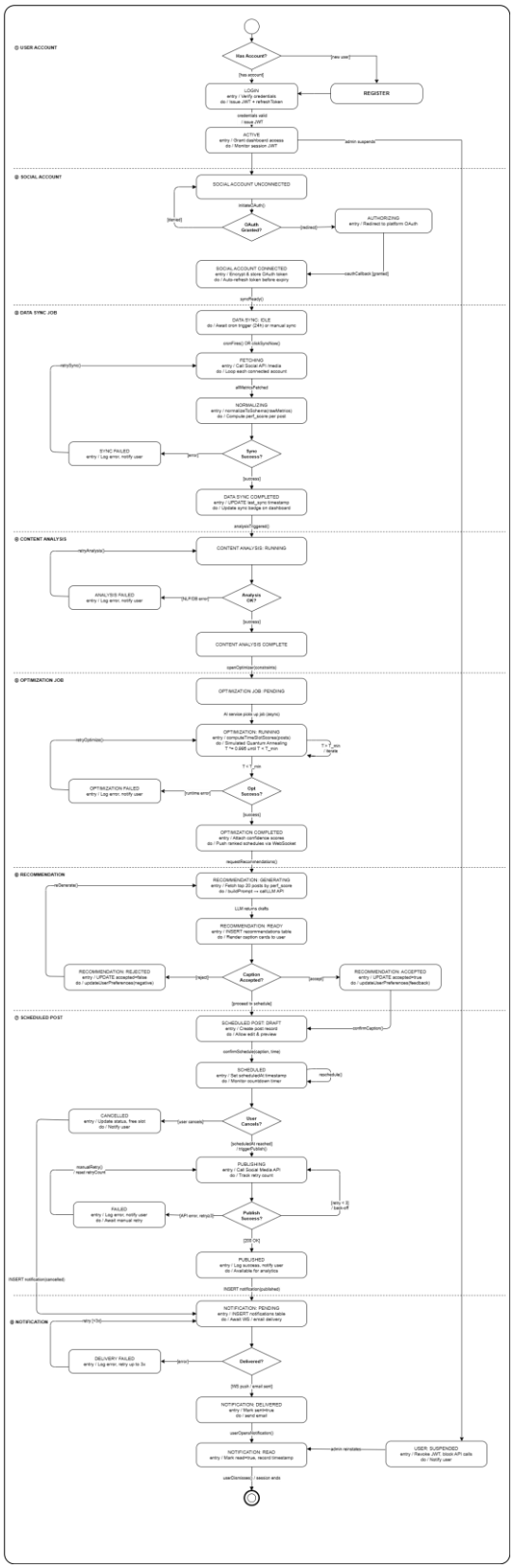


Figure 5-12 : State machine diagram

5.6 Algorithm Viewpoint

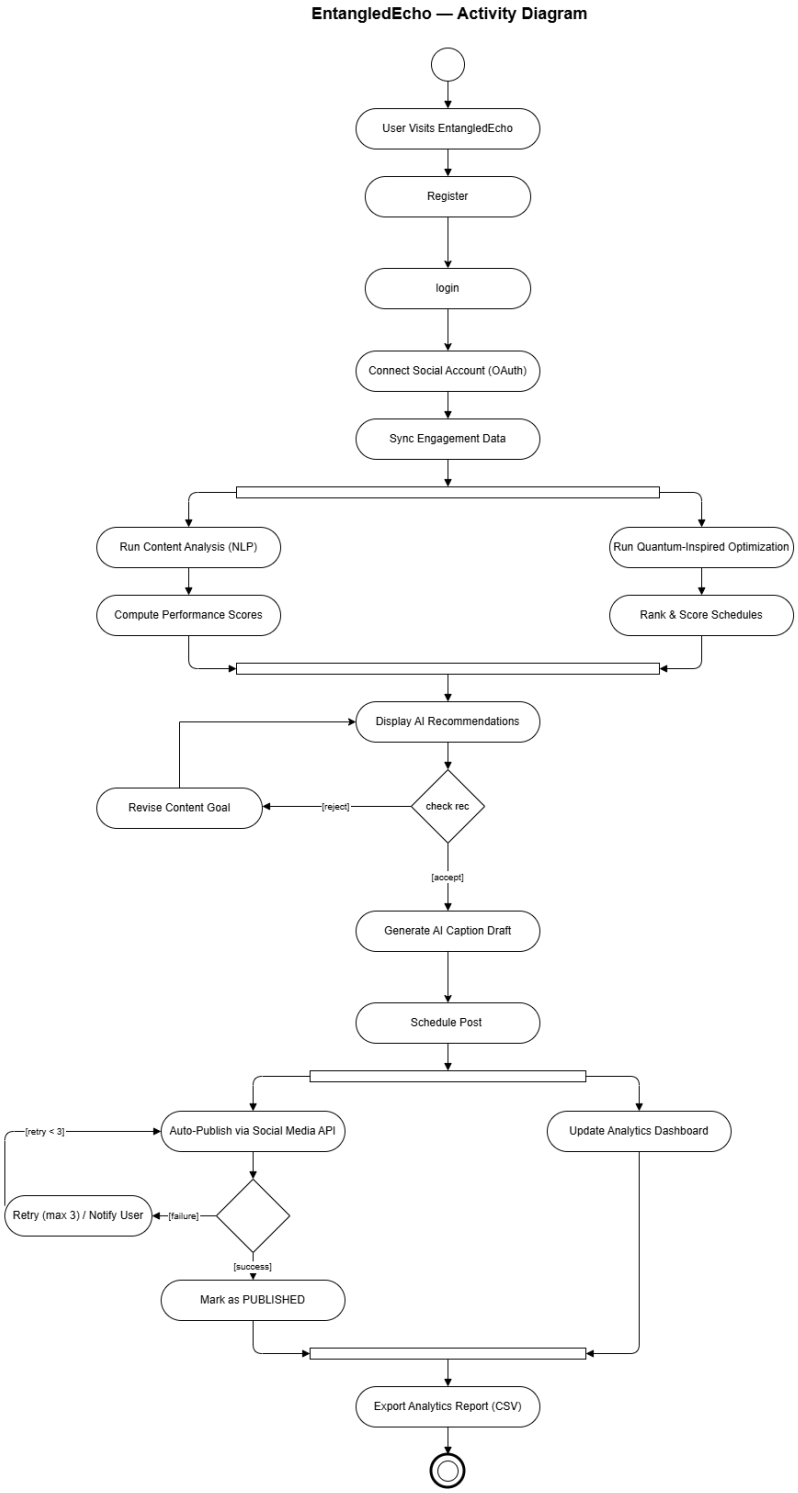


Figure 5-13 : Activity diagram

The following pseudocode describes the two core algorithms in EntangledEcho: the Quantum-Inspired Optimization Scheduler and the AI Recommendation engine.

5.6.1 Algorithm 1: Quantum-Inspired Schedule Optimizer

INPUT: account_id, horizon_days, constraintsOUTPUT: ranked_schedules[ ]1. FETCH historical posts for account_id (last 90 days)2. COMPUTE engagement_score for each (day_of_week, hour) time slot3. INITIALIZE candidate_solution as random valid schedule within constraints4. SET temperature T = T_initial (e.g., 1000)5. REPEAT until T < T_min: a. GENERATE neighbor_solution by swapping two random time slots b. IF neighbor_solution violates constraints: SKIP c. COMPUTE delta = score(neighbor_solution) - score(candidate_solution) d. IF delta > 0: candidate_solution = neighbor_solution ELSE: Accept with probability exp(delta / T) e. REDUCE T = T * cooling_rate (e.g., 0.995)6. STORE candidate_solution as primary_schedule7. GENERATE k=3 alternative schedules by perturbing primary_schedule8. ATTACH confidence_score to each schedule9. RETURN ranked [primary_schedule] + alternatives

5.6.2 Algorithm 2: AI Recommendation Engine

INPUT: account_id, recommendation_typeOUTPUT: recommendations[ ]1. FETCH top 20 posts by performance_score for account_id2. EXTRACT patterns: - top_media_types [ ] - top_hashtag_clusters [ ] - top_caption_topics (via TF-IDF / NLP) - top_posting_times [ ]3. IF recommendation_type == "caption": a. BUILD prompt from top_caption_topics + account niche b. CALL LLM API with prompt c. RETURN 3 caption drafts with hashtag sets4. IF recommendation_type == "schedule": a. LOAD optimization result from optimization_jobs table b. RETURN next 7 recommended slots with confidence_scores5. IF recommendation_type == "content_type": a. RANK media_types by mean performance_score b. RETURN ranked list with reasoning6. STORE recommendations in recommendations table7. RETURN recommendations with confidence_scores